

Reducing the Problems of Group Undo

Matthias Ressel
Advanced Engineering Center
UBS AG, Basel, Switzerland
matthias.ressel@ubs.com

Rul Gunzenhäuser
Faculty of Computer Science
University of Stuttgart, Germany
rul@informatik.uni-stuttgart.de

ABSTRACT

Providing undo functionality in groupware has been identified as an important, but difficult problem. Existing solutions show deficiencies like lacking generality, being too complex, being inefficient, or failing to yield acceptable results in common situations. In this paper we describe a new approach which reduces local group undo to a transformation-based method for combining the effects of concurrently issued user commands. Besides transformations we introduce mirror and folding operators in order to build a unique and consistent multidimensional model of the user interaction. In this way the problems of group undo mentioned above can be either overcome or at least reduced.

Keywords

CSCW, consistency maintenance, group editors, operation transformations, real-time groupware, undo.

INTRODUCTION

Undo is a common user request in single-user applications. The user wants to reverse inadvertently issued commands, back up from mistakes found later on, revert to former application states, or explore alternative solutions. Undo reduces the stress users are exposed to, helps them to be more satisfied and to work more efficiently. Undo is basically a user intention, that can be addressed either by explicit undo functions or by providing other means like inverse functions [1].

Switching to real-time groupware like group editors, users expect all the functionality known from single-user applications, including undo. In fact, undo proves even more important when working together with other people, because users are especially afraid of destroying artifacts created by other participants. The availability of undo in groupware will help to overcome such fears.

In groupware we distinguish *global* and *local group undo*, depending on whether a user undoes the last interaction step executed *globally*, no matter who issued it, or the last one issued by the *local* user.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

GROUP 99 Phoenix Arizona USA
Copyright ACM 1999 1-58113-065-1/99/11...\$5.00

Methods to implement undo known from single-user applications can also be used to implement global group undo, but they are not sufficient to implement local group undo. E.g., applying an inverse command to the current application state, may not yield the desired result, because commands executed by other users might have modified the application state in the meantime. This is an important reason why undo for groupware proves to be much more complex than undo for single-user applications. Several approaches to solve this problem have been proposed [1,2,7]. All of them record a linear interaction history containing commands from all participants. These approaches, however, lack generality, simplicity, awareness, correctness, or efficiency (see the discussion below).

This paper presents a new transformation-based approach to local group undo. It aims to show how a multidimensional representation of multi-user interaction can reduce the inherent complexity and can help to understand group undo better. It extends and clarifies our work described in a former paper [8]. In the following we first describe the main problems that may arise with local group undo. Then we infer five main requirements for local group undo. In the main part of the paper, we present our novel approach to local group undo that is based on the adOPTed-algorithm for combining the effects of concurrent user commands. Finally, we compare our solution with other approaches and discuss the gained experience.

PROBLEMS OF LOCAL GROUP UNDO

From a software designer's point of view, global group undo is not much different from common single-user undo schemes. It can be implemented by executing the inverse command – presumed to exist – for each user command to be undone. Global group undo always leads to a former application state. In contrast, local group undo is more complex because, in most cases, it does not lead to a former application state.

Example 1: Suppose user X appends the text *S* to a string *M*. Let us denote this command by *Ins*[2,*S*], where 2 is the position of the insertion (one-based) and *S* the text to be inserted. Then user Y inserts the text letter *A* in between by issuing the command *Ins*[2,*A*], the resulting string being *MAS*. Now, user X decides to undo her insertion command. What has to be done? A naïve, but incorrect

approach would be to execute $\text{Del}[2,S]$, i.e., the inverse command to $\text{Ins}[2,S]$, where 2 denotes the position of the deletion and S the deleted text. However, the current text MAS does not even contain an S at position 2. Deleting the second character at position 2 will delete the A instead and this is obviously not user X 's intention. The interaction steps can be illustrated as in Figure 1. Horizontal arrows denote commands from user X and vertical arrows commands from user Y . The boxed texts describe the application states. The correct command to execute would have been $\text{Del}[3,S]$, since the S has been moved to position 3. Note that the resulting text MA is an application state not occurring before, but the one expected.

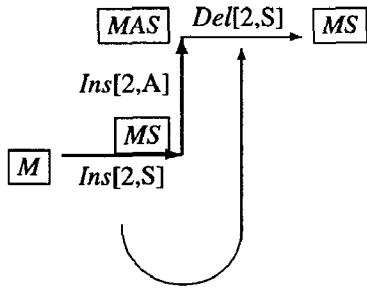


Figure 1: Incorrect naïve approach to local group undo

Obviously, returning to a previous application state cannot represent a criterion for local group undo to be considered correct. In addition, there are more situations where existing methods for group undo fail. One characteristic example is the following problem, which we call the Order Puzzle:

Example 2 (Order Puzzle): Suppose we start with a text buffer containing ab . Now user X first deletes a by issuing the command $\text{Del}[1,a]$. Afterwards user Y deletes b by issuing the command $\text{Del}[1,b]$. This will leave the buffer empty. Now both users undo their commands, respectively. What application state would you expect? Hopefully ab , because undoing all the commands should not modify the original application state effectively. Unfortunately, many undo schemes yield under certain conditions the wrong resulting state ba [7,8].

These observations led us to two basic questions related to local group undo:

1. Is it in general possible to undo a user command, if there have been other commands executed since? And if it is, *how* is it possible?
2. Do general criteria exist to state that a given method for local group undo is correct and will satisfy users' expectations?

Answers to these questions will follow later.

REQUIREMENTS

Based on our description of the problems in the previous section as well as considerations taken from user-centered design [5] and experiences with groupware in general, we identify five main requirements on local group undo: correctness, simplicity, efficiency, generality, and awareness.

Correctness: For group undo to be considered correct independently of the application, several principles must be satisfied:

1. The effect of an undo request may not depend on other users' requests. Especially, it may not depend on the moment a user issues an undo request, provided the same user has not issued any other commands in the meantime.
2. The user may return to any application state that occurred before by undoing all requests executed afterwards (for each participant individually).

Simplicity: In a very general group undo scheme, all users might be able to undo any command ever executed, no matter whether issued by the user herself or by any other participants. This includes the latest commands as well as commands executed anytime in the past. In this general case, users would have to specify for each undo request, which command they want to undo.

Typically, however, most users will undo only their own commands, and among these likely the last one executed or a sequence of the latest n commands. In most single-user applications users issue n undo commands in order to undo the effects of their latest n commands. Applying local group undo should be as simple as this.

Efficiency: It is intuitively clear, that in general a command to be undone has to be related with all commands executed afterwards. Therefore the complexity of local group undo is expected to be of linear order. But there is a special case where the complexity is expected to be lower. When the overall effect of all commands executed afterwards is a noop (a null operation), local group undo should be computable in constant time. Finally, the special case where a user tries to undo a certain number of his latest commands while other participants proceed with their work should be supported efficiently.

Generality: The undo functionality should always be available. Users should be able to trust the system that all their actions can be reversed. Otherwise the advantages of undo would vanish. Undo must be applicable to all user commands including a previous undo command (this would be called *redo*).

Awareness: Once a command is part of the interaction history, it should not be removed. Therefore an undo command should also be added to the interaction history

and not modify or delete any contained information. The design rationale behind this requirement is to keep the interaction history as complete as possible, especially in multi-user environments, where users rely on it to review and to understand the activities of other participants. A complete interaction history thus supports awareness. A consequence of this may be stated as a general rule: If in doubt, whether to keep information or not, it is better to keep it.

TRANSFORMATION-BASED APPROACH

Consistency Maintenance Problem

There has been some evidence that group undo is closely related to the problem of consistency maintenance [8]. To understand consistency maintenance it is useful to look at the requirements for real-time group editors. High responsiveness, high concurrency, high awareness, and independence of network latency – especially for geographically dispersed users in the Internet environment – have been identified as key requirements of such systems [9]. This leads to major design problems. E.g., for transatlantic connections the latency cannot be reduced below about 1/10 seconds due to the limits of the speed of light. High responsiveness can then only be achieved if the application state, e.g., a text document, is replicated and if user commands are executed on the local copy without delay. The so-called optimistic algorithms [4] follow this approach. Optimistic algorithms have to make sure, however, that the following three properties are not violated [9]:

1. Convergence: The final document states have to be equal on all sites.
2. Causality Preservation: The execution order of user commands has to follow the cause-effect order.
3. Intention Preservation: The actual effect of an executed command has to equal the intended effect.

These subproblems can be summarized as the consistency maintenance problem.

Multi-Dimensional Interaction Model

In order to solve the consistency maintenance problem we have found out that representing the interaction of n users as a grid-based diagram or, more formally, as an n -dimensional interaction model, can help to visualize concurrent interaction of multiple users [8]. It helped to find deficiencies in an earlier solution to the consistency maintenance problem, the dOPT algorithm (distributed Operational Transformation algorithm [3]), which did not use a multi-dimensional representation. And it led to an improved version which we called the adOPTed algorithm (adapted dOPT for editors) [8].

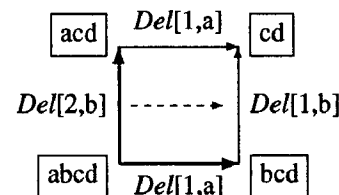
The Transformation-Based AdOPTed Algorithm

As already stated, in optimistic algorithms for consistency maintenance the application state is replicated and user

commands have eventually to be applied to each copy. If all users start with an identical application state it can be expected that – after several user commands have been issued – the applications states will be again identical again after every command will have been applied. This is just the convergence property already mentioned above. With an optimistic approach, local commands are privileged and will be executed first followed by commands received from remote sites. Thus every site may execute a sequence of user commands in a different order. As can be seen easily, executing the unmodified commands will be either impossible or will lead to diverging and unexpected application states, thus violating the properties of convergence and intention preservation. However, appropriate transformations of incoming remote requests can avoid this divergence and can preserve the intended effect of a user command.

The adOPTed algorithm uses a set of application-dependent transformation rules to guarantee convergence and intention preservation. Causality is preserved by ensuring that the execution order (which is a total ordering of all user commands since every site can execute commands only sequentially) specializes a partial ordering defined by the causal relationship of the user commands.

An example illustrated in Figure 2 will help to understand the transformation rules.



Y's command Del[2,b] has to be shifted to the left when it will be executed after X's Del[1,a]

Figure 2: Example of Transformation Rule

Assume the starting state of the group editor is *abcd*. With the command Del[1,a] user X deletes the first character. With the command Del[2,b] user Y concurrently deletes the second character. In state *bcd* user X executes the command from user Y. It has to be taken into account, that the local command Del[1,a] has been executed. Therefore user Y's delete command has to be transformed from Del[2,b] to Del[1,b] (illustrated by the dashed arrow in Figure 2) in order to delete the character *b* as intended. On the other hand, user Y may execute user X's command Del[1,a] without modification. Note that without this transformation user X would have deleted the second and wrong character *c*, resulting in an unintended application state *bd* as well as an application state different from *cd* on user Y's site.

The labeled arrows are called requests. They contain more information than commands, like the site they were issued, a serial number and a description of the state they were issued in. More formally a request is a quadruple (u, k, v, c) , where u denotes the user, k the serial number, v a state vector of length n and c the command. This information is implicitly contained in the diagrams. If a state is described by the state vector $v = (v_1, v_2 \dots v_n)$ then the execution of the first v_u of each user u led to this state. The serial number is redundant since it always equals $v_u + 1$, but it is convenient to denote user requests by the combined unique key (u, k) . It is also convenient to define accessor functions $u(r)$, $k(r)$, $v(r)$, and $c(r)$, which access the components of a request. Two requests are called *congruent*, if only their vector states differ.

Typically for every pair of requests – or rather command classes – there will be a transformation rule. As an example, if any command is transformed in respect to a noop command, no modification of the commands will be necessary. When we apply these transformation rules iteratively, we will gain a grid-based diagram of requests. Its dimension will equal the number of participants. It can be proven, that, no matter which path we take from one point to another, executing the commands along the way will lead to the same application state. An important part of the proof is that the commands attached to the vertices will be unique. This is trivial for only two participants, but needs some more insight for three or more participants. In this way, in distributed real-time group editors convergence between replicated application states can be enforced [8]. An appropriate definition of the transformation rules also ensures that the intention is preserved. The next sections describe how this method can be extended to implement local group undo.

TRANSFORMATION-BASED LOCAL GROUP UNDO

Single-step Local Group Undo

First we will answer the question, how a local group undo function can be realized, which reverts just *one* local command.

Suppose we want to undo our last command after a command from another participant has already been executed. For correctness of local group undo, we required that the effect of the undo request does not depend on other users' requests. Therefore, we may safely assume, that we issued the undo command immediately after the execution of the command which is to be undone. Then the appropriate transformations can be applied to gain the command that has to be executed in the present state.

Example 3: Suppose, user X appends the text S to the starting state M by issuing command $Ins[2,S]$. After user Y has inserted the text A with $Ins[2,A]$ user X decides to undo her command. Now, we may safely assume, that user X might have issued her undo request already when the

application state was MS . At that time the undo is easily achieved by applying the inverse command $Del[2,S]$. We call this procedure of inserting the inverse command *mirroring*. It is illustrated in Figure 3. The command $Del[2,S]$ is transformed in respect to user Y's command $Ins[2,A]$ to yield the command $Del[3,S]$, which is obviously the expected solution.

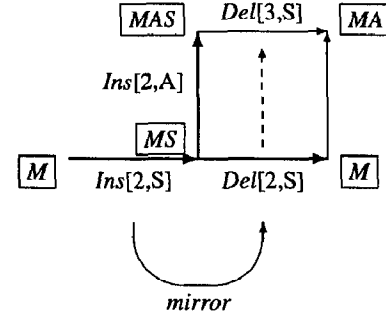


Figure 3: Local undo by transformation

Mirror operator

The mirror operation can be formally defined by the following procedure:

```
mirror(r, j):  mirror r for j-th undo in row
              u'=u(r);
              v'=incr(v,u,2j-1);
              c'=inverse(c);      // inverse command
              return (u',k+(2j-1),v',c');
```

```
incr(v, u, i): // increment u-th comp. by i
              v'=copy(v); // make a copy of v
              v'[u]=v'[u]+i; // increment u-th comp.
              return v'
```

Unfortunately, the computation of requests in the interaction model by transformation and by mirroring is not sufficient to solve the Order Puzzle in Example 2. Here is another example that would fail.

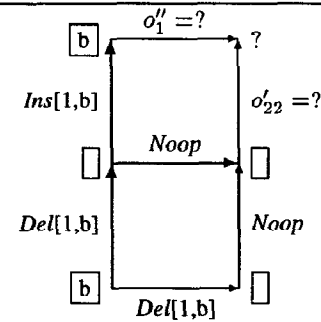


Figure 4: Del-del-undo puzzle

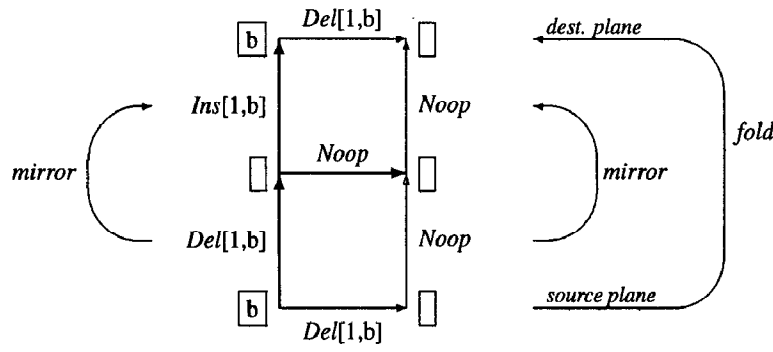


Figure 5: Application of mirror and fold operators

Example 4 (del-del-undo-puzzle): Suppose both users X and Y simultaneously delete the single character b (see Figure 4). The corresponding transformation rule transforms both delete commands to noop commands, since deleting the same character just once is sufficient. Now suppose, user Y wants to undo his command $\text{Del}[1,b]$. We know this can be achieved by applying the mirror operator, adding the command $\text{Ins}[1,b]$. How can the missing commands be computed? By applying the transformation rule the commands would just be copied without modification, e.g., another noop command would lead from state b to the end state b (in the upper right corner). However, this obviously violates our second principle of correctness, namely that a set of do-undo-pairs like $\text{Del}[1,b]$ and $\text{Ins}[1,b]$ should have no effect on other users' commands. If this were to be satisfied, the resulting state should be empty, as after just executing user X's command $\text{Del}[1,b]$.

Folding operator

To solve the puzzle in Example 4, we observe that all parallel lines (or in general the planes) connected by do-

undo-pairs should actually contain congruent requests and states. Here in the example o_1 should be $\text{Del}[1,b]$ and the resulting state should be empty, like the command and the state at the bottom of the diagram, respectively (see Figure 4). We denote the copying of a request from the beginning of a do-undo-pair – or more general a set of possibly nested do-undo-pairs – to the end of the do-undo pair as *folding*. This illustrates the fact that we might actually fold the interaction model – here along the straight line defined by the horizontal noop request in the center – so that both planes come to lie above each other. To complete the interaction model correctly we demand that o'_{22} will be computed by mirroring the command noop (in Figure 4 at the bottom right). The resulting interaction model with all mirror and fold operators applied is displayed in Figure 5.

Folding can be formally defined as follows:

```
fold(r, w, j):    //
                  // precondition: u(r) ≠ w
                  v' = incr(v, w, 2j);
                  return (u, k, v', c);
```

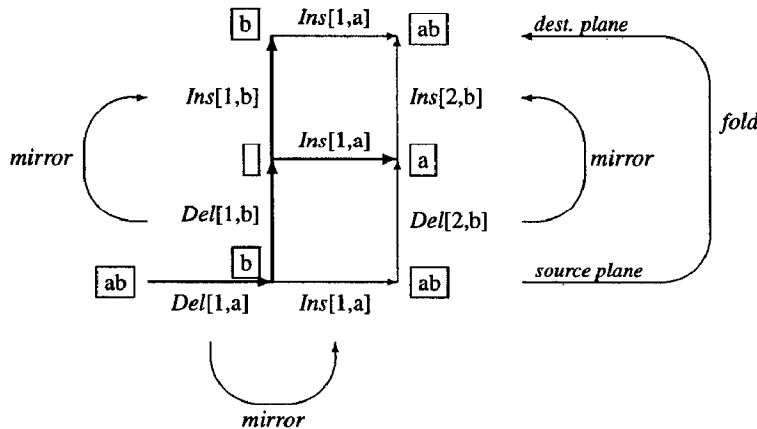


Figure 6: Solution to the order puzzle

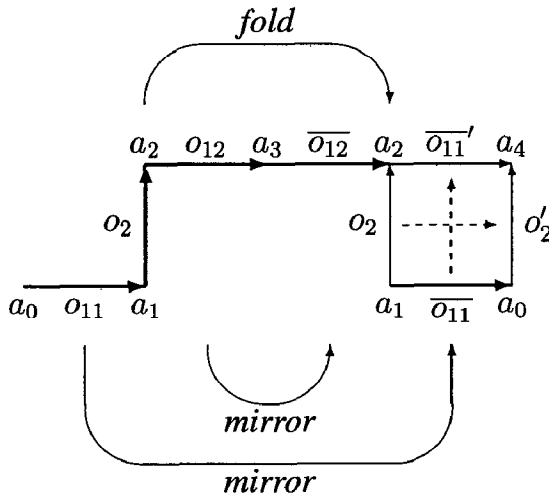
The interaction model for the Order Puzzle in Example 2 can be computed with the same operators. The desired result is shown in Figure 6.

Multiple-step Undo

The procedure for single-step undo can also be applied to multiple-step undo. To compute the interaction model a combination of transformations, mirror and folding operators has to be applied.

Example 5: Figure 7 illustrates a situation, where user X has first executed her command o_{11} , then user Y his command o_2 , and finally user X her command o_{12} . Now X wants to undo both of her commands. In order to undo the last command o_{12} , the mirror operation is applied, resulting in the inverse command of o_{12} . To undo command o_{11} , the mirror operator is applied again. In order to be able to transform the resulting request into the application state a_2 , user Y's command o_2 has to be copied by applying the folding operator. Finally, the transformation can be computed to complete the interaction model.

Note that all pairs of requests and application states symmetrical to the vertical folding plane passing the state a_3 are actually congruent.



$a_0, a_1, \dots$ denote application states

Figure 7: First mirror/fold, then transform

Example 6: Figure 8 illustrates a situation resembling Example 5. Here users X and Y issued their first commands concurrently. After both commands have been executed, user X issues a further command o_{12} . Like in Example 5, applying the mirror operator reverses the command o_{12} . However, this time the undo of o_{11} can be computed by applying the mirror operator as well.

Note that all pairs of requests symmetrical to the vertical folding plane that are orthogonal to the folding plane are inverse to each other while all other request and

application states symmetrical to the vertical folding plane are congruent.

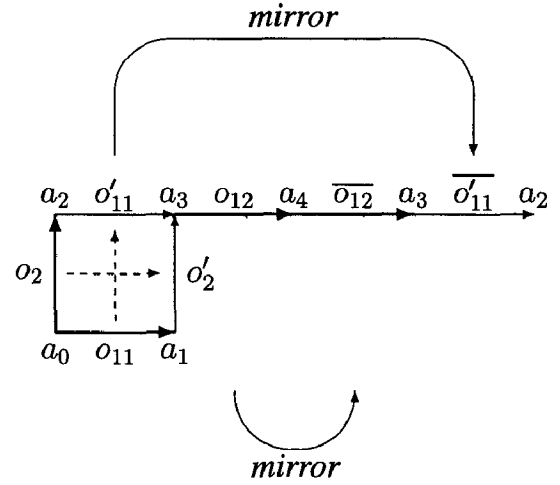


Figure 8: First transform, then mirror/fold

ALGORITHM

The algorithm for local group undo is a rather simple extension to the adOPTed algorithm. The main function of the adOPTed algorithm *translates* a request into the currently valid state vector. Appendix A shows the revised version of this function together with a new one that computes the request necessary to undo a user request r in the current state v when this forms the j -th undo command in a row.

Correctness

Our undo approach relies on the correctness of the applied transformation process. The correctness of the adOPTed algorithm has been proven under the assumption that certain properties are satisfied by the transformation rules. Basically it has to be shown that the command labeling a vertex of the interaction model is unique and does not depend on the order in which the interaction model is constructed and computed. This can be proven by mathematical induction over the distance of a vertex from the origin. First the skeleton of user requests seeded into the interaction model is certainly unique. Second, it can be shown that all transformations from unique vertex labels lead to unique vertex labels again. To generalize this proof to our algorithm for local group undo it is sufficient to show that all command labels remain unique when mirror and fold operators are applied. In fact it can be seen easily from the algorithm, that a new request is computed either by transformation, by mirroring, or by folding. From the definition of the fold and mirror operations it follows that paired requests on both sides of and parallel to the symmetrical axis of a mirror and folding operation are congruent. This ensures that do-undo-pairs anywhere in the interaction model do not affect the rest of the interaction model. In fact, the whole space

between a source plane and a destination plane of do-undo-pairs could be eliminated from the interaction model if the users don't need the information any more.

Complexity

The cost of computing the request necessary to undo a request from the past mainly depends on the number of requests other users have issued between the past request and now. For each of those requests usually one transformation has to be executed. If this set of requests from other users contains do-undo-pairs they do not count since they may be skipped by using the fold operator. So the actual cost will be constituted by one mirroring operation plus at most as many transformations as there have been requests from other people afterwards. So the complexity is somewhere between constant in the best case, when all the other requests form do-undo-pairs, and linear, when there are no do-undo-pairs at all. Typically, local undo is issued shortly after some commands have been executed, so the real cost will be rather low. The storage mainly depends on the number of transformations, so the space complexity is about proportional to the time complexity.

EVALUATION

The proposed method for local group undo has been implemented in the group editor Joint Emacs [8] running on SUN workstations under the X Window System. This editor has been successfully tested in a real-world setting as a discussion and brainstorming tool. The user interface for the undo functionality is a rather simple one, as known from the single-user Emacs editor. Users may either execute a command by typing *Alt-x undo*, they can use the predefined shortcut *Ctrl-x u*, they can hit the undo button available on SUN keyboards, or they may bind the command to whatever key they want. No visualization of the interaction model is available. Users do not notice the underlying group undo scheme at all, they just behave and feel as if they were using the undo function of a single-user application. Actually this can be considered as a big success since it proves a smooth and seamless transition between single-user and multi-user usage in that respect.

Oscillation

Oscillation may occur, if users try to correct the same error simultaneously. E.g., two users might both insert a missing word. Realizing that it has been inserted twice, both of them issue an undo command resulting in the original state. Now both reinsert the missing word and so on. This is a common experience also found in the real life (e.g. if two persons meet on a sidewalk trying to pass each other). Usually there will be some other communication channel for coordination. There may be some social protocol, determining, e.g., who has currently the role of a text writer and who will do just spelling correction. In fact, during our user tests, occurrences of oscillation were very rare.

DISCUSSION

Is undo always available?

The answer to this question is "Yes, if..." If transformation rules exist for every pair of user commands, it is in fact possible to undo local commands. The given approach depends on finding such transformation rules. Rules for simple text editing have been found. Finding transformation rules for other applications is still a matter of research. Another limitation of our approach seems to be that it is only possible to undo local commands chronologically, but it is not possible to undo any command. In fact, chronological undo is the one most users are familiar with. It is what they expect. We don't claim we have a solution for implementing selective undo. However, our approach gives some new insight. Imagine that not only every user is assigned a new dimension of the interaction model, but that every single command is assigned a new dimension. Every dimension would then be reserved for a command and all following undo or redo commands. Theoretically this would solve the selective undo problem. However, it would require defining the interaction model with a dimension higher than the number of commands to be issued. This number would be rather big. It can be shown that the complexity of the algorithm is quadratic in respect to the number of sites. So under real conditions it seems not feasible to implement selective undo by our scheme. However, we may imagine a humbler solution: We might assign 10 dimensions for each user that we use for her commands in turn. Then it would be possible to undo selectively one of the last ten local commands executed.

Is the undo correct?

It is actually difficult to define necessary and sufficient conditions for an undo scheme to be correct. However, we identified two necessary conditions. First, if we undo some command – or more general, if we undo a whole set of executed commands – we expect no overall effect on the application state. In fact, by introducing the mirror and fold operators our undo scheme satisfies this requirement easily. Second, it can be expected that the moment when such an undo command is issued does not effect the result. In fact, this requirement has been utilized in the transformation-based design of our algorithm; so it is satisfied, too.

Can the results be transferred to other environments?

The presented core algorithm for local group undo is application-independent, as is the algorithm for consistency maintenance. However, the approach relies on the fact that – for a given application – appropriate transformation rules are found. So far, we have only experience with transformation rules for text editing functions like insert and delete. Here we can it is possible to show that undo can indeed be offered for every user command, no matter whether it is overlapping with

another one or not. This experience can be generalized, however, to any application that works on linear data structures with objects being inserted and deleted. E.g., in graphical editors we have the problem of the stacking order of drawing objects. We are about to incorporate our algorithms into a graphical editor to find out if this will be helpful in practice. It has already been shown elsewhere that transformation-based approaches can also be applied to spreadsheet applications [6].

How does this algorithm compare to other approaches?

GINA [2] uses an approach called direct undo, which is actually not restricted to collaborative applications. Each command is represented as a command object. Each object class has methods to apply, reapply, and undo the command to the current application state. A command can only be undone, if the application state matches the state after the original execution of the command. Each command object has a method that tests, if the command can be undone. Example: A square has been colored red. The effects of this command can only be undone, if the square is still colored red in the current application state. In this way undo is easily implemented, but is not always available.

In DistEdit [7] transformations are used to reorder a linear command history in order to move the command that is to be undone to the end of the history. Do-undo-pairs in the history, that prevent the application of a transformation, have to be eliminated first. Transformations are undefined and therefore not applicable, if user commands modify regions next to each other or overlapping. Therefore, the applicability of the DistEdit undo scheme is less general than our scheme. The resulting complexity for local group undo is of square order in respect to the length of the interaction history. This worst case arises, if all of the following commands form do-undo-pairs. This relates to the constant time we need in our approach. DistEdit did not provide a solution to the order puzzle. Not too surprisingly, it has been found that the transformations used in DistEdit can be reduced to the transformations used in our concurrency algorithm adOPTed [8].

Abowd and Dix [1] propose the usage of pointers as placeholders for deleted characters in a text in order to keep their respective order when reinserted by undo. This approach has the disadvantage, that an increasing list of pointers has to be updated with every further user command, even if the pointers will not be needed later at all.

Earlier we proposed a predecessor to our transformation-based method described here [8]. It was the first undo scheme based on the adOPTed-algorithm. However, first approach needed an artificial construct of a temporary virtual user to be able to compute the needed transformations within the interaction model. This made

it hard to prove any kind of correctness. Now, restricting undo to the most common cases of chronological undo for each user allows us to build the undo commands into the interaction model permanently. The introduction of mirror and folding operators helped in achieving and proving the correctness of our new approach in the sense given above.

CONCLUSION

We have presented a new application independent approach to local group undo, that meets the previously identified requirements generality, awareness, simplicity, correctness, and efficiency. It supports the capability of undoing any type of command including an undo command itself. It is easy to use, because users do not have to specify explicitly, which command they want to reverse. It is correct because it satisfies two necessary conditions we have identified. It supports awareness by keeping information once it is added to the interaction model. Finally, it is efficient for the most frequent cases, the order of complexity being linear in the general case and constant if the interaction history only consists of blocks of do-undo-pairs. This has been achieved by representing multi-user interaction as a multi-dimensional interaction history.

ACKNOWLEDGMENTS

This article is the result of a cooperation between the Advanced Engineering Center of the UBS AG in Basel, Switzerland, and the Institute for Computer Science at the University of Stuttgart, Germany.

ADDRESS

Current address for Matthias Ressel: Institute for Computer Science, University of Stuttgart, Breitwiesenstr. 20-22, D-70565 Stuttgart, Germany, ressel@acm.org.

REFERENCES

1. Abowd, Gregory D., and Dix, Alan J. Giving Undo Attention. *Interacting with Computers*, 4(3), 317-342, 1992.
2. Berlage, Thomas. A selective Undo Mechanism for Graphical User Interfaces Based on Command Objects. *ACM Transactions on Computer-Human Interaction*, 1(3), 269-294, September 1994.
3. Ellis, C.A., Gibbs, S.J. Concurrency control in groupware systems. *Proceedings of ACM SIGMOD Conference on Management of Data*, pp.399-407. 1989.
4. Ellis, C.A., Gibbs, S.J., Rein G.L. Groupware: Some issues and experiences. *CACM* 34(1), pp. 39-58, January 1991.
5. Norman, Donald A., Draper, Stephen W. (eds.). *User Centered Systems Design: New Perspectives on Human-Computer Interaction*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1986.

6. Palmer, Christopher R., Cormack, Gordon V. Operation Transforms for a Distributed Shared Spreadsheet. *Proceedings of the CSCW '98*, Seattle, WA, November 14-18, 1998.
7. Prakash, Atul, and Knister, Michael J. A Framework for Undoing Actions in Collaborative Systems. *ACM Transactions on Computer-Human Interaction*, 1(4), 295-330, December 1994
8. Ressel, Matthias, Nitsche-Ruhland, Doris, and Gunzenhäuser, Rul. An Integrating Transformation-Oriented Approach to Concurrency Control and Undo in Group Editors. *Proceedings of CSCW'96*, Boston, Mass., November 16-20, 1996, 288-297.
9. Sun, Chengzheng, Ellis, Clarence (Skip): Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements. *Proceedings of the CSCW '98*, Seattle, WA, November 14-18, pp. 59-68.

Appendix

TranslateRequest translates a request r from its own state vector $v(r)$ to a given state vector v :

```

translateRequest(r, v):
    If exists  $u \neq u(r)$  where  $r(u, v_u)$  was an undo request then
        let the undo request be the  $j$ -th in a row;
         $v' = \text{incr}(v, u, -2j)$ ;
        return fold(translate( $r, v'$ ),  $j$ );
    else if exists  $u \neq u(r)$  where reachable( $v'$ ) with  $v' = \text{incr}(v, u, -1)$  then
        return transform(translateRequest( $r, v'$ ),
        translateRequest( $r(u, v_u), v'$ ));

```

GetUndoRequestInState computes the request that has to be executed in state vector v in order to undo request r :

```

getUndoRequestInState(r, v, j):
     $U = u(r)$ ;
     $V' = \text{incr}(v, u, 2j-1)$ ;
    If  $v(r) = v'$  then
        return mirror( $r, j$ );
    else if reachable( $v'$ ) then
        return mirror(translateRequest( $r, v'$ ),  $j$ );
    else
        return translateRequest(mirror( $r, j$ ),  $v$ );

```

Reachable and *transform* are function defined by the original adOPTed algorithm. *Reachable* tests if it is possible to reach a certain state vector. *Transform* is the function that actually applies transformation rules to command requests.